题目：

[云计算（期末复习题含答案） 云计算习题-CSDN博客]

[自评丨这几道华为云计算认证基础题答对了吗 - 知乎 (zhihu.com)]

[华为ICT大赛-云赛道-云计算初级 华为ict云赛道题库-CSDN博客]

知识点：

[云计算——内存虚拟化与I/O虚拟化-CSDN博客]

[云计算 虚拟化 网络豆的博客-CSDN博客https://blog.csdn.net/weixin_43909650/category_12098799.html)]

云计算
云计算感觉每年考的可能都不太一样，而且要看和什么“品牌方”合作，像前几年，老师说上的是AWS的内容，这几年上的是华为的内容，但是今年好像和华为的合作就要到期了，之后也不知道会不会再签了。
上届的情况
根据上一届的学长说，他们考试考的是选择题和填空题，里面都会有1~2道的华为云相关的内容。选择题考过CDN和一些常见的缩写，最后一道大题是一个流程图，描述这个流程图干了什么，据说挺简单的。
这届的情况
题型：
我们这次考试呢题型是15道单选题，一题两分，7道多选题，一题三分，还有5道简答题，一题六分，后面还有四道大题，分数分布没仔细看。
内容：
关于内容嘛，感觉分布还挺均匀的，把课程群里的四个PPT和一篇文档看完就差不多了。如果课程群里有题目，一定要去刷一刷，我考前和室友说了压缩包里有题目，然后室友都去看了，考试的时候，选择题一半以上都是一样的，多选题完全一样！所以！！！一定要把题目过一下！！！我这次考试真的立大功！！！还有就是华为云认证的相关知识也可以看看。这次考试的内容嘛，单选题大多都是华为和云计算的基础知识，还考了三道AWS的题目，还是英文的？？？其中有一道是AWS的职责啥的。
简答题第一题是画实验一的架构（就是hardware、windows、vmware、hardware、Vmware esix、linux、app那个架构）；
第二题是资源池化，问了资源池化是什么，有什么好处；
第三题是分布式计算，问这是什么东西；
第四题是全虚拟和半虚拟的异同；
第五题是弹性伸缩。
大题第一题是物联网和云计算的结合及应用，第二题是IaaS、SaaS、PaaS叫什么，应用于什么场景，第三题是云的三个服务类型，第四题是内存虚拟化。
如果老师没有发题目，且想要这次题目的（就是我上面说很多重复的，全部都是选择题）可以私信我，我单独发给你，因为老师说课程相关的资料不要随便传hh。

一、云计算基础概念

云计算

云计算是一种模型，它可以实现**随时随地、便捷地、随需应变地**从**可配置计算资源共享池**中获取**所需的资源**（例如，网络、服务器、存储、应用、及服务），资源能够**快速供应并释放**，使管理资源的工作量和与服务提供商的**交互减小**到最低限度。

通过**互联网**可以使用足够强大的计算机为用户提供的服务， 这种服务的使用量可以使用**统一的单位**来描述。

优势：

1. 按需自助服务
2. 广泛的网络接入
3. 资源池化
4. 快速弹性伸缩
5. 可计量服务

资源池化

将物理或逻辑上的资源集中管理，形成可动态分配的资源池，以满足不同用户或应用的需求。这种做法可以有效提升资源的利用，同时提供快速弹性和动态伸缩的能力。

资源池化能够实现**按需自助服务**，通过将所有资源**分解到最小单位**，可以**屏蔽不同资源的差异性**，提升资源的**利用率**，简化系统管理，实现资源**整合**，使IT对业务的变化更具适应力。

物联网和云计算的结合及应用

物联网（IoT）和云计算的结合在当今的科技领域中展现出了巨大的潜力和价值。这种结合不仅促进了数据的收集、存储和处理，还推动了智能化、自动化的发展，为各行各业带来了前所未有的变革。以下是物联网和云计算结合的主要应用及其特点：

一、数据存储与处理

1. 物联网设备**生成海量的数据**，这些数据可以**通过云计算平台进行集中存储和管理。**云计算提供了**强大的存储和计算能力**，可以满足物联网设备**大规模数据处理的需求。**
2. 云计算提供了灵活的**资源调度**方式，可以**根据实际需求动态分配计算资源**，提高数据处理效率。

二、边缘计算

1. 为了降低数据传输时延和费用，边缘计算将一部分计算任务从云端转移到物联网设备附近的网关或边缘节点进行处理。这种结合方式可以在保证实时性的同时，减少对带宽的依赖。
2. 边缘计算与云计算协同工作，形成**云-边协同的计算模式**，为**物联网应用提供高效、可靠的计算服务。**

三、协同控制与管理

1. 云计算提供了**中心化的协同控制和管理平台**，通过与物联网设备连接，实现**对设备的集中监控**、调度和管理。
2. 用户可以通过云平台对物联网设备进行远程配置、升级和故障排除等操作，提高设备的维护和管理效率。

四、数据分析与挖掘

1. 云计算提供了强大的数据分析和挖掘能力，可以**对物联网设备的数据进行深入分析**，发现隐藏的规律和价值。
2. 利用数据分析结果，可以实现**更精准地控制和管理物联网设备**，优化业务流程，并提供个性化的应用和服务。

五、应用拓展与创新

1. 云计算为物联网应用提供了**灵活的平台**，使得开发者可以**快速构建和部署各种应用。**
2. 通过云计算，物联网应用可以实现快速迭代、灵活扩展，从而为用户提供更好的体验和服务。

物联网和云计算的结合应用已经深入到各个领域之中，如智能制造、智慧城市、智能家居、智能医疗等。在智能制造领域，物联网设备可以**实时采集数据并上传至云端**，通过云计算进行数据分析和优化，实现生产过程的**智能化和自动化。**在智慧城市领域，借助**物联网传感器**和云计算技术，可以对城市基础设施、交通、环境等进行**实时监测和管理**，提高城市管理效率和服务水平。

总之，物联网和云计算的结合为各行各业带来了前所未有的变革和机遇。随着技术的不断进步和应用场景的不断拓展，这种结合将在更多领域得到广泛应用，并推动社会的科技进步和经济发展。

边缘计算与雾计算

边缘计算是一种**分散式运算**的架构，将应用程序、数据资料与服务的运算，由网络**中心节点**移往网络逻辑上的**边缘节点**来处理。边缘计算将原本完全由中心节点处理大型服务加以**分解**，切割成更小与更**容易管理**的部分，分散到边缘节点去处理。　核心价值**CROSS。**

雾计算 即本地化的云计算， 将云计算无缝扩展到边缘， 利用并依靠**传统的网络设备组件收集信息**、 处理后发送回**需要数据的设备**；如处理能力放在 IoT设备所在的局域网里面， 由网络内的IoT网关提供服务。【单个设备接收多个终端来的信息，处理后发回】。特点 低延时 位置感知精确 地理分布广泛 移动性高。

CDN

内容分发网络。将网站的内容发布到最接近用户的网络"边缘"，使用户可以就近取得所需的内容，解决Internet网络拥挤的状况，提高用户访问网站的响应速度。

分布式计算

边缘计算也是一种分布式计算。

分布式计算是一种计算模式，其中计算资源（如处理器、存储和网络）分布在多个物理或虚拟的计算机上，这些计算机通过网络相互连接，并共同处理一个任务或应用。

资源分散到各个节点上、节点之间通过通信协作完成计算任务、可以轻松地添加或删除节点来适应不同的计算需求、由于存在多个节点，一个节点的故障通常不会导致整个系统崩溃。

网格计算

网格计算是一种特殊的分布式计算，它将地理位置分散的**异构资源**（包括计算、存储、数据、仪器等）整合成一个**统一的计算平台**，以**支持大规模、复杂的科学计算和数据处理任务。**通过云端，可在**特定的时段组合出特定目的组合和使用庞大的计算机网格**，节省购买和部署资源带来的开支。跨多台计算机拆分任务，显著减少处理时间。

与并行计算不同，网格计算项目通常没有与之关联的时间依赖关系。它们**仅在闲置时才使用网格中的计算机**，而且操作员可随时执行与网格无关的任务。

并行计算

并行计算是一种同时执行多个计算任务或任务的一部分的计算模式，以提高计算速度和效率。这些任务或任务的部分可以在同一台计算机的不同处理器上执行，也可以在网络中的多台计算机上执行。

多个任务或任务的部分可以同时执行、并行计算系统通常具有共享的内存或数据存储空间、通过并行执行，可以显著提高计算速度和效率。

微服务

微服务架构风格是一种将**单个应用程序**作为一套小型服务开发的方法， 每种应用程序都在自己的进程中运行， 并与**轻量级机制**（通常是HTTP资源API） 进行通信。 这些服务是**围绕业务功能**构建的， 可以通过全自动部署机制独立部署。

优点：独立部署、灵活拓展、资源有效隔离、团队组织架构的优化调整

缺点：项目拆分增加了开发和测试难度；不同服务之间的数据不一致性难度大

无服务器

Serverless是一种构建和管理基于微服务架构的完整流程， 允许你在**服务部署级别**而不是服务器部署级别来管理你的应用部署。

Faas函数即服务，服务商提供一个平台，允许客户开发、运行和**管理应用程序功能**，而**无需构建和维护**基础架构。

Baas后端即服务，服务商为移动应用开发者提供整合云后端的边界服务。用户不再关注后端，更关注函数逻辑

FaaS特点：无需运维、聚焦代码、快速启动生命周期短、无穷弹性计算、自动触发

Baas优点：提高效率、缩短上市时间、减少交付APP所需资源、针对手机和平板优化，安全和弹性的基础设备、大量常用的API资源。

二、虚拟化技术

虚拟化

虚拟化的本质就是将原先的物理设备进行**逻辑化**， 转化成一个文件夹或文件， 实现**软硬件的解耦。**

分类：

裸金属型虚拟化（此时的VMM也为Hypervisor）、宿主型虚拟化

特点：

分区、隔离、封装、独立

CPU虚拟化-全虚拟和半虚拟

相同点：

两者都依赖于**虚拟机管理程序（Hypervisor）**来创建和管理虚拟机。Hypervisor是介于物理硬件和虚拟机之间的软件层，**负责提供对物理资源的虚拟化访问。**两者的根本目的都是为了实现虚拟化，即在一台物理机上模拟出多个虚拟环境，使得每个虚拟环境看起来像独立的物理环境一样。

不同点：

完全虚拟化是一种全面**模拟硬件**的虚拟化方式，它允许多个虚拟机在同一台物理机上运行不同的操作系统，每个虚拟机都可以独立运行，仿佛在独立的物理服务器上运行一样。在完全虚拟化中，虚拟机的操作系统并不知道它是在虚拟机上运行的，因为虚拟机的硬件和操作系统看起来和物理机的硬件和操作系统没有区别。

完全虚拟化可以在不对真实硬件进行任何更改的情况下运行，并且能够支持传统的操作系统，例如 Windows，但是由于需要进行硬件仿真，完全虚拟化通常会导致性能下降。

【VMM处于ring0层，Guest OS处于ring1层，Guest OS不知道自己处于虚拟环境，全虚拟化的技术：1. 陷入模式和模拟仿真（Trap-and-emulation），OS的特权指令由VMM自动捕获后模拟再返回OS（有问题，x86的部分敏感指令不属于特权指令，VMM会捕获特权指令，则导致可能的敏感指令不会被捕获）； 2. 二进制翻译技术，也是一种编译，从一种机器代码翻译成另一种机器代码，有解释执行、动态翻译、静态翻译等方法】

半虚拟化则是一种虚拟化方式，它要求**操作系统修改**以支持虚拟硬件，并且在修改后的操作系统上运行虚拟化软件。在半虚拟化中，虚拟机能够通过访问虚拟化层提供的接口（Hypercall）来与物理硬件交互，而无需通过硬件仿真的方式进行。

由于不需要进行硬件仿真，因此半虚拟化通常比完全虚拟化的性能更高。但是，要求虚拟机操作系统修改，可能并不适用于某些操作系统，例如 Windows。

【Guest OS 的部分代码被改变，从而使 Guest OS会将和特权指令相关的操作都转换为发给 VMM 的 Hypercall（超级调用），由 VMM 继续进行处理。 类似于给Guest OS提供了一个“系统调用”，把它无权执行的指令交给ring0的VMM来执行】

GPT：

1. 对操作系统的要求：
    - **全虚拟化**：不需要对操作系统进行任何修改，操作系统认为它正在运行在真实的硬件上，而实际上它是在虚拟机管理程序提供的模拟环境中运行。这种方式的优点是提供了良好的**隔离性**，减少了潜在的安全风险和性能损失。
    - **半虚拟化**：要求操作系统进行一定的修改，以适应虚拟机管理程序提供的**半虚拟化API**。操作系统需要进行一些修改，以识别并利用虚拟化环境中的特殊指令和接口。这种方式可以提高虚拟机的运行效率，但可能会**降低兼容性和可移植性。**
2. 性能效率：
    - **全虚拟化**：由于不需要修改操作系统，全虚拟化提供了**最彻底的隔离。**然而，由于全虚拟化需要**频繁的上下文切换**（即虚拟机管理程序和虚拟机之间的切换），这可能会影响性能表现。此外，全虚拟化通常比半虚拟化更耗费资源。
    - **半虚拟化**：通过修改操作系统并减少隔离程度，半虚拟化可以提高虚拟机的运行效率。它减少了上下文切换的频率，从而提高了性能。然而，半虚拟化可能不支持未经修改的操作系统（如Windows系统），这限制了其适用范围。
3. 兼容性：
    - **全虚拟化**：由于不需要修改操作系统，全虚拟化具有更好的兼容性。任何可以运行在裸机上的软件（通常是操作系统）都可以未经修改地运行在虚拟机中。

- **半虚拟化**：由于需要修改操作系统，半虚拟化的兼容性较差。它可能不支持某些未经修改的操作系统或应用程序。

4. 适用场景：
   - **全虚拟化**：适用于需要高度隔离和兼容性的场景，如**多租户环境、云计算平台**等。
   - **半虚拟化**：适用于需要提高性能并愿意接受一定兼容性限制的场景，如**特定的应用服务器**、数据库服务器等。

KVM

KVM（Kernel-based Virtual Machine）是一种基于Linux内核的开源虚拟化技术，它允许在单个物理服务器上创建和运行多个虚拟机。

KVM是Kernel-based Virtual Machine的缩写，是Linux内核中的一个虚拟化模块。自Linux 2.6.20版本后，KVM集成在Linux的各个主要发行版本中，并成为学术界和工业界广泛使用的虚拟机监视器（VMM）之一。

QEMU

QEMU是一种通用的开源计算机仿真器和虚拟器。QEMU共有两种操作模式

全系统仿真：能够在任意支持的架构上为任何机器运行一个完整的操作系统

用户模式仿真：能够在任意支持的架构上为另一个Linux/BSD运行程序

同时当进行虚拟化时，QEMU也可以以接近本机的性能运行KVM或者Xen。
与KVM的关系：QEMU经常与KVM一起使用，以提供完整的虚拟化解决方案。在这种情况下，QEMU提供**虚拟机管理和设备模拟**等功能，而**KVM提供硬件虚拟化支持**。QEMU利用KVM来提高虚拟机的性能，特别是通过CPU的虚拟化扩展。

Xen

Xen是一个基于X86架构的开源虚拟化技术

Libvert

它是一套开源的API、管理工具,用来管理虚拟化平台。

概念区别

QEMU是一个开源的虚拟化工具，提供了完整的虚拟化功能，并经常与KVM一起使用以提高性能。

KVM是一个Linux内核模块，提供了硬件虚拟化支持，并与QEMU结合使用以提供完整的虚拟化解决方案。

Xen是一个基于X86架构的开源虚拟化技术，支持半虚拟化和全虚拟化技术

内存复用

内存共享：内存共享是一种在多个虚拟机之间共享同一物理内存空间的技术。虚拟机之间通过共享物理内存区域来实现内存的复用。当虚拟机需要对共享内存进行写操作时，系统会开辟另一内存空间，并修改映射关系，确保数据的一致性和隔离性。

内存气泡：通过Hypervisor（虚拟机监视器）的监控，当检测到某个虚拟机内存使用率较低时，Hypervisor会主动回收该虚拟机暂时不用的物理内存，并释放给内存使用率较高的虚拟机。这个过程对虚拟机上的应用是透明的，即应用无感知。 通过内存气泡技术，物理内存可以被更高效地利用，减少了内存资源的浪费。

内存置换：内存置换是一种将虚拟机长时间未访问的内存内容置换到存储（如硬盘）中的技术。

常规软件内存虚拟化

GVA→GPA→HVA→HPA

从连续的GPA到非连续的HVA由 VMM 定义的**映射表**（由数据结构 kvm_memory_slot记录）完成

影子页表技术

传统的内存虚拟化方式，虚拟机的每次内存访问都需要 VMM 介入 ，影子页表则简化了这个过程，实现**GVA到HPA的直接映射。**

必须为 Guest 的系统页表设计**一套对应的影子页表**，然后将影子页表装入 **Host 的MMU** 中，这样当 Guest 访问 Host 内存时，就可以根据 MMU 中的影子页表映射关系，完成 GVA 到HPA 的直接映射。而**维护这套影子页表的工作则由 VMM** 来完成。 【每个进程都有自己的虚拟地址空间，意味着VMM要给Guest中的每个进程页表都维护一个影子页表，本质上还是纯软件，维护工作太多】

EPT技术（硬件辅助内存虚拟化）

EPT 在原有 CR3 页表地址映射（GVA到GPA）的基础上，引入了 EPT 页表来实现另一层映射（GPA到HPA），这样， GVA->GPA->HPA 的两次地址转换都由硬件来完成。

内存虚拟化过程

内存虚拟化的本质就是把物理机的真实物理内存**统一管理**，包装成多份**虚拟的内存**给若干虚拟机使用。保证每个进程都被赋予一块**连续的虚拟内存空间。**

内存虚拟化的核心，在于引入一层新的地址空间---guest OS物理地址空间，guest OS以为自己运行在真实的物理地址空间中，实际上它是通过VMM访问真实的物理地址的。在VMM中保存客户机地址空间和物理机地址空间之间的映射表。

硬盘虚拟化

硬盘阵列构成存储池并进行逻辑划分，通过**虚拟化文件系统**进行分配

【硬盘阵列→存储池（物理卷）→逻辑卷→虚拟化文件系统→共享目录】

RAID：RAID由几个硬盘组成， 从整体上看相当于由多个硬盘组成的一个大的物理卷。 在物理卷的基础上可以按照指定容量创建**一个或多个逻辑单元**， 这些逻辑单元称作**LUN（Logical Unit Number）** ，可以作为映射给主机的基本块设备。 【相当于逻辑卷中的基本单元是LUN】

RAID(独立硬盘冗余阵列)是一种将多个同类型硬盘通过一定的数据组织方式组成容量更大、 安全性更高、 性能更好的存储的技术。

分布式存储：不同于集中式存储有独立存储设备， 分布式存储通常是通过软件将多台服务器的硬盘整合成一个存储资源。

块存储与文件存储：

- 块存储：也称为卷存储，是一种为虚拟机提供独立磁盘空间的技术。它将磁盘空间划分为多个独立的块，每个块都可以被单独分配给虚拟机。
- 文件存储：是一种将文件系统中的文件和目录以文件的形式存储在磁盘上的技术。它关注的是文件和目录的组织和管理，而不是数据的块划分。

从块存储和文件存储性能差异的角度解释为什么虚拟机拷贝要比创建一个新的虚拟机要快。

从块存储和文件存储性能差异的角度来解释为什么虚拟机拷贝要比创建一个新的虚拟机要快，可以归纳如下：

1. 存储机制差异：
    - 块存储：在块存储中，数据被组织成**固定大小的块**（block），这些块被**直接映射**到物理磁盘空间。块存储通常用于**高性能应用**，因为它提供了低延迟和高吞吐量的磁盘访问。
    - 文件存储：文件存储将**数据组织成文件和目录**，并通过**文件系统**（如NFS、FTP等）进行访问。文件存储的**读写速率通常低于块存储**，因为每次读写操作都需要通过文件系统层进行解析和处理。
2. 虚拟机拷贝过程：
    - 虚拟机拷贝通常涉及将虚拟机的**磁盘映像**（通常是一个或多个**vmdk**文件）从源位置复制到目标位置。由于**这些文件是块存储的映像**，它们可以直接在**物理磁盘上进行高效的复制操作。**
3. 新虚拟机创建过程：
    - 创建一个新的虚拟机通常包括多个步骤，如**分配磁盘空间、格式化磁盘、安装操作系统和配置环境**等。这些步骤中的许多都涉及到**文件存储**的操作，特别是当使用基于文件的虚拟机映像（如OVA、OVF）进行部署时。
    - 特别是安装操作系统和配置环境这一步，需要执行**大量的磁盘读写操作**，这些操作在文件存储环境中可能会受到性能限制。
4. 性能差异：
    - 虚拟机拷贝主要依赖于**块存储的高效磁盘访问能力**，可以快速地将虚拟机的磁盘映像从一个位置复制到另一个位置。
    - 而创建新的虚拟机则需要通过文件存储系统执行更多的磁盘读写操作，这些操作可能会受到文件系统层性能开销的影响，导致整体速度较慢。
5. 总结：
    - 虚拟机拷贝之所以比创建一个新的虚拟机要快，主要是因为虚拟机拷贝**主要依赖于块存储的高效磁盘访问能力**，而创建新虚拟机则需要执行更多的文件存储操作，这些操作可能会受到文件系统层性能开销的影响。因此，在需要快速部署虚拟机的情况下，使用虚拟机拷贝通常是一个更高效的选择。

网络虚拟化

（VLAN就是一种网络虚拟化，在原有网络基础上通过VLAN Tag划分出多个广播域）

网络虚拟化保障我们创建出来的虚拟机可以正常通信、访问网络

【传统网络架构：台物理主机包含一个或多个网卡（NIC），要实现与其他物理主机之间的通信，需要通过自身的 NIC 连接到外部的网络设施，如交换机上。1）是某些应用大部分情况可能处于空闲状态，2）是当应用增多的时候，只能通过增加物理设备来解决扩展性问题。不管怎么样，这种架构都会对物理资源造成极大的浪费。 】

将一张物理网卡虚拟成多张虚拟网卡（vNIC），通过虚拟机来隔离不同的应用。

【可以解决上述问题：1），可以利用虚拟化层 Hypervisor 的**调度技术**，将资源从空闲的应用上调度到繁忙的应用上，达到资源的合理利用；针对问题 2），可以根据物理设备的资源使用情况进行**横向扩容**，除非设备资源已经用尽，否则没有必要新增设备。 】

其中**虚拟机与虚拟机之间**的通信，由虚拟交换机完成，**虚拟网卡和虚拟交换机之间**的链路也是虚拟的链路，整个**主机内部构成了一个虚拟的网络**，如果虚拟机之间涉及到三层的网络包转发，则又由另外一个角色— —虚拟路由器来完成。

总结下来，网络虚拟化主要解决的是虚拟机构成的网络通信问题，完成的是**各种网络设备的虚拟化**，如网卡、交换设备、路由设备等。

虚拟化特性

负载均衡：用集群的负载均衡特性动态调整虚拟机所属主机， 保障主机资源利用率尽量一致，提高系统可靠性。

易于扩容

内存复用

按模板部署虚拟机与批量复制

资源热添加与虚拟控制台

快照

OpenStack

OpenStack是一个开源的云计算**管理**平台，由多个软件开源项目组合而成。

OpenStack是开源云计算平台， 可**控制**整个数据中心的大型计算，存储和网络**资源**池。

用户能够通过**Web界面、命令行或API接口**配置资源。

容器技术

传统Paas项目的痛点：应用打包困难（因本地环境与云端环境不一致 ）

容器镜像打包了应用及其依赖（包含完整操作系统的所有文件和目录） ，包含了应用运行所需要的所有依赖 。容器镜像核心在于实现**应用及其运行环境整体打包**以及打包**格式统一**。实现本地环境与云端环境的高一致性。

容器是容器image运行时的**实例**

	容器	虚拟机
启动速度	秒甚至毫秒启动	数秒至数十秒
系统内核	共享内核	不共享内核
实现技术	利用Linux内核技术Namespace/Cgroup等实现。	依赖虚拟化技术实现，由Hypervisor层实现对资源的隔离
隔离效果	进程级别的隔离	系统资源级别的隔离
资源消耗（性能）	容器中的应用只是宿主机上的一个普通进程	使用虚拟化技术，就会有额外的资源消耗和占用
资源调用（敏捷性）	应用进程直接由宿主机OS管理	应用进程需经过Hypervisor的拦截和处理，才能调用系统资源
运行数量	一台服务器上能启动1000+容器	一台服务器上一般不超过100台虚拟机
应用	DevOps、微服务等	用于硬件资源划分
镜像	分层镜像	非分层镜像

容器编排

支持应用程序的容器的部署和组织称为容器编排， 这是通过容器编排工具完成的。 包括调度、资源管理、服务发现、自动伸缩等。

Cluster ： 是计算、 存储和网络资源的集合。

Master 是 Cluster 的大脑，它的主要职责是**调度**，即决定将**应用放在哪里运行。** Master 运行 Linux 操作系统，可以是物理机或者虚拟机。为了实现高可用，可以运行多个 Master。

Node 的职责是**运行容器应用。** Node 由 Master 管理， Node 负责**监控并汇报容器的状态**，并根据 Master 的要求**管理容器的生命周期。** Node 运行在 Linux 操作系统，可以是物理机或者是虚拟机。

Pod是 Kubernetes 的**最小工作单元。**每个 Pod 包含**一个或多个容器。** Pod 中的容器会作为一个整体被 Master 调度到一个 Node 上运行。

Controller：Kubernetes 通常不会直接创建 Pod，而是**通过 Controller 来管理 Pod** 的。 Controller 中定义了 Pod 的部署特性，比如有几个副本，在什么样的 Node 上运行等。

Kubernetes Service 定义了**外界访问一组特定 Pod 的方式。** Service 有自己的 IP 和端口， Service 为 Pod提供了负载均衡。

运行Pod：Controller； 访问Pod：Service

三、华为云的服务模式与云服务

弹性伸缩与负载均衡

弹性负载均衡（ Elastic Load Balance） 通过将访问流量自动分发到多台弹性云服务器， 扩展应用系统对外的服务能力， 实现更高水平的应用程序容错性能。

负载均衡方案：

在VPC/子网内创建弹性**负载均衡实例**作为**服务接口对外提供服务**；

实例内创建**监听业务端口**的**监听器**，配置监听器的**后端服务器组**，以及**流量分发策略** ；

将**负责业务处理的服务器**加入到对应的**后端服务器组**中

协议：TCP UDP HTTP HTTPS

策略：轮询、最少连接、源IP

弹性伸缩（Auto Scaling）可根据用户的**业务需求和预设策略**，自动调整**计算资源**或**弹性IP资源**，使云服务器数量或弹性IP带宽自动随业务负载增长而增加，随业务负载降低而减少，节省云上业务资费，保证业务平稳健康运行。

典型应用场景	**场景描述**
Web应用服务	常见 Web 服务的逻辑层服务器扩缩容。 如企业网站、电商、视频网站、在线教育、移动应用等，客户端的请求通过负载均衡到达应用服务器。当访问量快速变化时，弹性伸缩服务可根据请求量弹性扩缩应用服务器的数量。若您使用了伸缩带宽功能，弹性伸缩服务也可根据访问流量多少自动调整IP公网带宽大小。
高性能计算集群部署	常见 Web 服务的分布式后台扩缩容。 如分布式大数据计算的计算节点、数据检索服务器等后端计算集群，根据计算量大小实时调整集群服务器数量。
请求类服务器部署	用于发送请求或收集数据的服务器集群的部署。 此类服务有明显的时效性，可依靠弹性伸缩服务快速完成请求服务器的创建部署和容量的扩大或缩小。

弹性伸缩创建流程：

1. 创建伸缩组（最大/最小实例数，期望实例数，负载均衡）
2. 创建伸缩配置（使用模板创建）
3. 创建伸缩策略（告警策略、定时策略[基于配置的某个时间点，618]、周期策略）【执行动作：增加、减少、设置为】

IaaS、SaaS、PaaS的应用场景/产品实例

IaaS（Infrastructure as a Service）：以服务形式提供基于**服务器**和**存储等硬件资源**的可高度扩展和按需变化的IT能力。

如虚拟机ECS、裸金属服务器BMS、存储资源（云硬盘EVS、弹性文件对象SFS、对象存储服务OBS、内容分发网络CDN、数据快递服务DES、云备份CBR）

EVS云硬盘（Elastic Volume Service）：提供持久性块存储的服务。

对象存储服务（Object Storage Service）是一款稳定、安全、高效、易用的云存储服务，具备标准Restful API接口，可存储任意数量和形式的非结构化数据。

网络类：VPC虚拟私有云，为云服务器、云容器、云数据库等资源构建隔离的、用户自主配置和管理的虚拟网络环境

弹性公网IP（EIP）：统一的公网接入方案

PaaS（Platform as a Service）：它提供给终端用户基于互联网的应用开发环境，包括应用编程接口和运行平台等，并且支持应用从创建到运行整个生命周期所需的各种软硬件资源和工具。

如RDS关系型数据库（Relational Database Service）：是一种基于云计算平台的即开即用、稳定可靠、弹性伸缩、便捷管理的在线关系型数据库服务。

CCE云容器引擎

SaaS（Software as a Service）：这些SaaS提供的应用程序减少了客户安装和维护软件的时间和技能等代价，并且可以通过按使用付费的方式来减少软件许可证费用的支出。

如华为云DevCloud：

IaaS（基础设施即服务）、SaaS（软件即服务）和PaaS（平台即服务）是云计算中的三种主要服务模式，它们各自在不同的应用场景中发挥着重要作用。

1. IaaS（基础设施即服务）

   应用场景：

   - 大规模数据处理和存储：如**科研机构进行大数据分析**，需要**弹性扩展的计算和存储资源。**
   - 在线游戏和多媒体服务：需要**处理大量并发用户请求**，确保低延迟和高可用性。
   - 灾难恢复和备份：企业可以使用IaaS来**备份关键数据**和运行关键业务应用，以防意外情况导致的数据丢失或服务中断。
   - 开发和测试环境：开发人员可以**快速部署和撤销虚拟环境**，以进行测试和实验。

2. SaaS（软件即服务）

   应用场景：

   - 客户关系管理（CRM）：如Salesforce，企业可以使用SaaS提供的CRM服务来管理客户信息、销售机会和服务请求。
   - **企业资源规划**（ERP）：如Oracle NetSuite，SaaS ERP解决方案**帮助企业整合和管理财务、供应链、人力资源等业务流程。**
   - **办公自动化**：如Microsoft Office 365或Google G Suite，**员工可以通过SaaS服务访问电子邮件、文档编辑、协作工具等。**
   - **在线学习和培训**：教育机构和企业可以使用SaaS提供的在线学习平台来提供课程和培训。

3. PaaS（平台即服务）

   应用场景：

   - 移动应用**开发**：开发者可以使用PaaS提供的**移动应用开发框架和工具**来快速构建和部署移动应用。
   - Web应用**开发**：PaaS提供**Web应用开发所需的服务器、数据库、中间件等资源**，以及开发工具、部署和管理功能。
   - **大数据分析**：PaaS平台可以提供大数据处理和分析所需的计算、存储和分析工具，帮助用户从数据中提取有价值的信息。
   - DevOps：PaaS支持持续集成、持续部署和自动化测试等**DevOps实践**，提高软件开发的效率和质量。

总的来说，IaaS、SaaS和PaaS各自在云计算领域发挥着重要作用，为不同的应用场景提供了灵活、高效和可扩展的解决方案。